



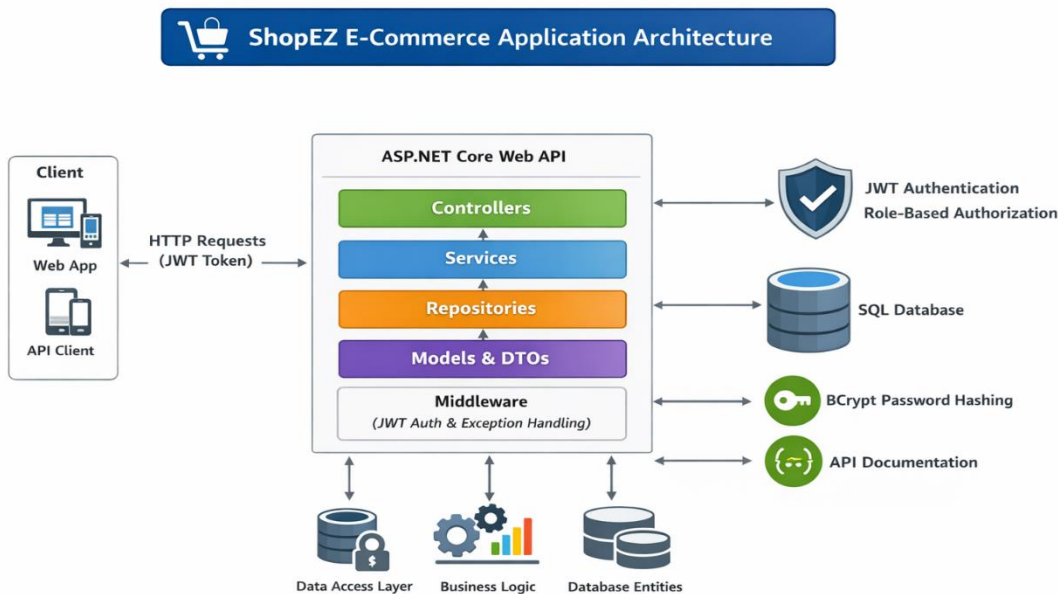
ShopEZ – E-Commerce Application

A scalable and secure E-Commerce Backend API built using ASP.NET Core Web API. This project implements authentication, product management, and order processing using JWT-based security and role-based authorization.

Features

- JWT Authentication & Authorization
- User Registration & Login
- Product Management (Admin only)
- Order Management System
- Admin User Management
- Global Exception Handling Middleware
- Swagger API Documentation

Architecture



Architecture Overview

The project follows a **Layered Architecture** for better maintainability and scalability:

- **Controllers** → Handle HTTP requests and responses
- **Services** → Contain business logic

- **Repositories** → Handle database operations
 - **Models** → Represent database entities
 - **DTOs** → Transfer data between layers
 - **Middleware** → Handle global exceptions
-

Technologies Used

- ASP.NET Core Web AP
 - Entity Framework Core
 - SQL Server
 - JWT Authentication
 - JWT Authentication
 - Swagger (API Testing)
-

Authentication & Authorization

JWT Authentication

- Users receive a JWT token after login
- Token must be included in every request

Authorization: Bearer <token>

User Roles

Admin

- Create, update, delete products
- View all orders
- Access all users

Customer

- Place orders
 - View own orders only
-

Project Structure

Controllers/ → API endpoints

Services/ → Business logic

Repositories/ → Data access layer

Models/ → Database entities

DTOs/ → Data transfer objects

Middleware/ → Exception handling

docs/ → Architecture diagrams

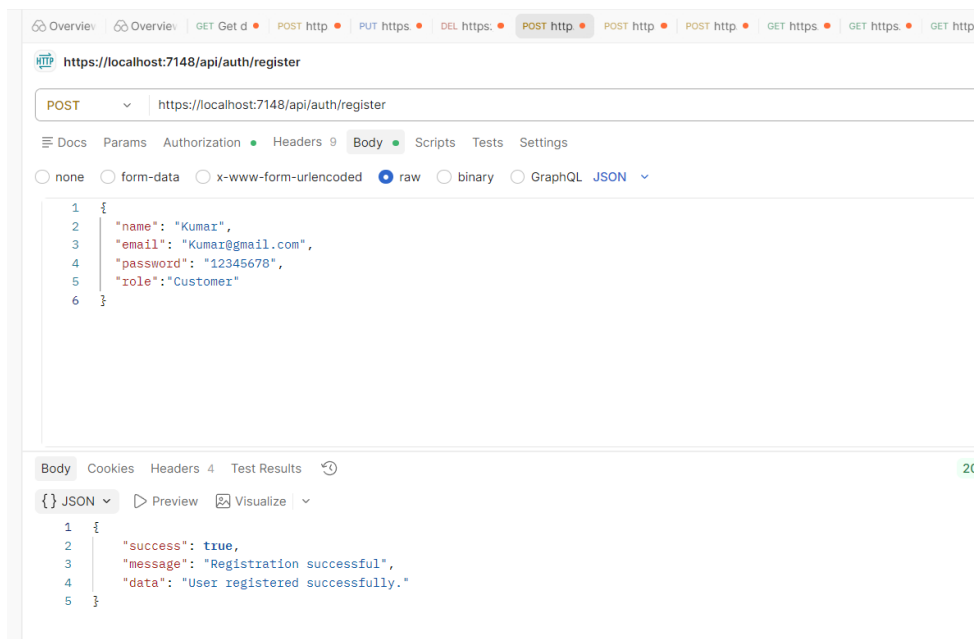
API MODULES

AUTH APIs

Register User

POST : <https://localhost:7148 /api/Auth/register>

```
{  
  "name": "kumar",  
  "email": "kumar@gmail.com",  
  "password": "12345678",  
  "role": "Customer"  
}
```

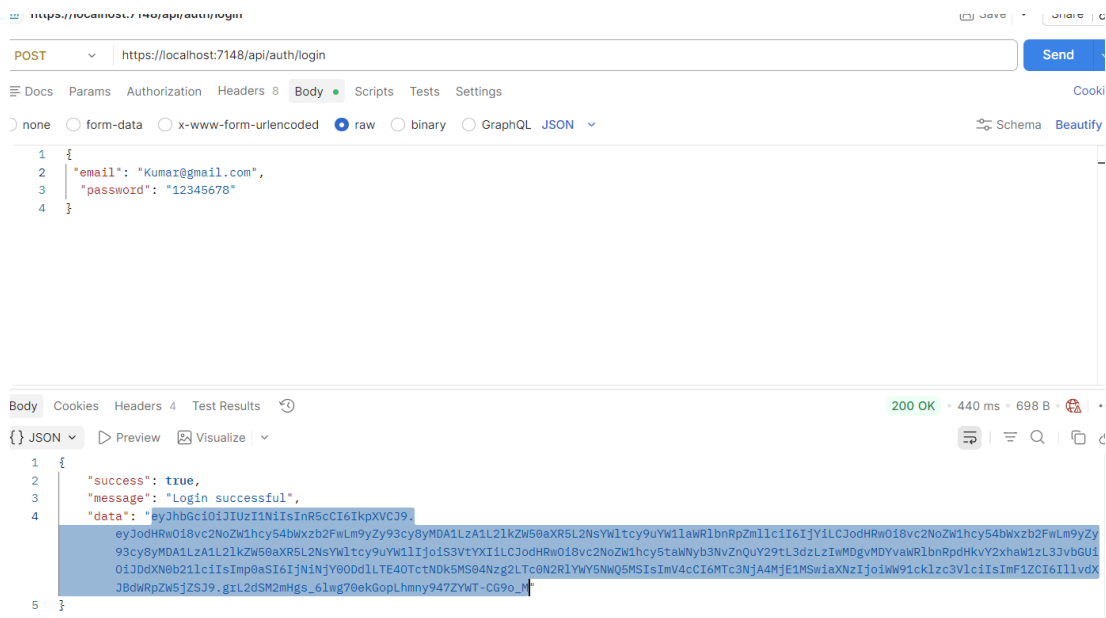


Login User

POST : <https://localhost:7148 /api/Auth/login>

```
{  
  "email": "john@gmail.com",  
  "password": "12345678"  
}
```

Response → JWT Token

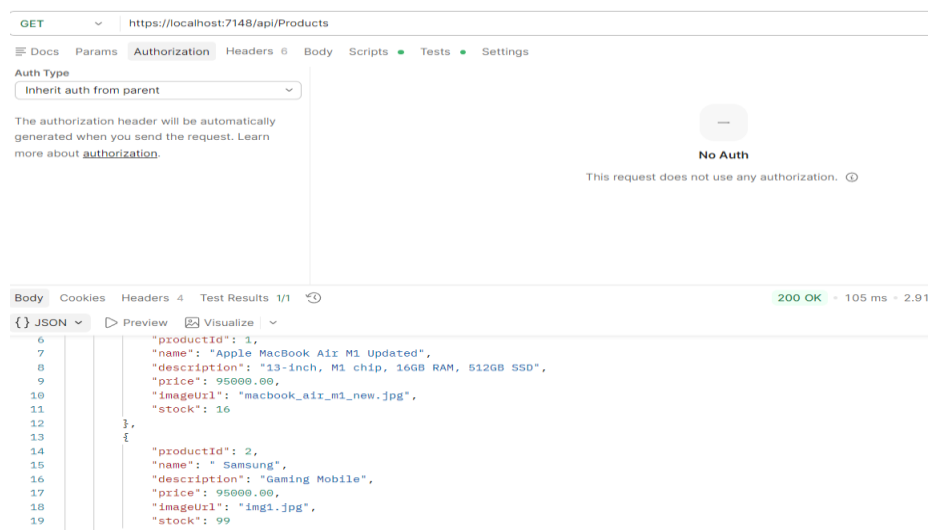


PRODUCT APIs

Method	Endpoint	Access
GET	/api/Products	Public
GET	/api/Products/{id}	Public
POST	/api/Products	Admin
PUT	/api/Products/{id}	Admin
DELETE	/api/Products/{id}	Admin

Get All Products

GET: [https://localhost:7148 /api/Products](https://localhost:7148/api/Products)



Get Product by ID

GET: <https://localhost:7148 /api/Products/{id}>

My Collection > Get data

GET <https://localhost:7148/api/Products/5>

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Auth Type
Inherit auth from parent

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#).

No A
This request does not use

Body Cookies Headers 4 Test Results 1/1

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Success",
4   "data": {
5     "productId": 5,
6     "name": "Sony WH-1000XM4",
7     "description": "Noise Cancelling Wireless Headphones",
8     "price": 25000.00,
9     "imageUrl": "headphones.jpg",
10    "stock": 18
11  }
12 }
```

Create Product (Admin)

POST: <https://localhost:7148 /api/Products>

<https://localhost:7148 /api/Products>

POST <https://localhost:7148/api/Products>

Docs Params Authorization Headers 9 Body Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "name": "Dolo 650",
3   "description": "Pharmacy",
4   "price": 65,
5   "imageUrl": "pharmacy.jpg",
6   "stock": 120
7 }
8
9 }
```

Body Cookies Headers 4 Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Product created",
4   "data": {
5     "productId": 23,
6     "name": "Dolo 650",
7     "description": "Pharmacy",
8     "price": 65,
9     "imageUrl": "pharmacy.jpg",
10    "stock": 120
11  }
12 }
```

Update Product (Admin)

PUT: [https://localhost:7148 /api/Products/{23}](https://localhost:7148/api/Products/{23})

The screenshot shows a REST client interface with the URL `https://localhost:7148/api/Products/23`. The HTTP method is set to **PUT**. The **Body** tab is selected, showing a JSON payload:

```
1 {
2   "name": " pack of Dolo650",
3   "description": "Pharmacy",
4   "price": 650,
5   "imageUrl": "pharmacy.jpg",
6   "stock": 120
7 }
8 }
```

Below the request, the **Body** tab shows the JSON response:

```
1 {
2   "success": true,
3   "message": "Product updated successfully",
4   "data": "OK"
5 }
```

Delete Product (Admin)

DELETE : [https://localhost:7148 /api/Products/{id}](https://localhost:7148/api/Products/{id})

The screenshot shows a REST client interface with the URL `https://localhost:7148/api/Products/23`. The HTTP method is set to **DELETE**. The **Authorization** tab is selected, showing the **Auth Type** set to **Bearer Token**. A text box labeled **Token** contains a masked token: `.....`. Below this, a note states: "The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization."

Below the request, the **Body** tab shows the JSON response:

```
1 {
2   "success": true,
3   "message": "Product Deleted successfully",
4   "data": "OK"
5 }
```

ORDER APIs

Place Order

POST: <https://localhost:7148/api/Orders>

`{"cartItems": [ {"productId": 1,"quantity": 2} ]}`

https://localhost:7148/api/Orders

POSThttps://localhost:7148/api/Orders

DocsParamsAuthorizationHeaders 9BodyScriptsTestsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

1{2"cartItems": [3{ "productId": 20, "quantity": 1 },4{ "productId": 19, "quantity": 1 }5]6}

BodyCookiesHeaders 4Test Results

{ }JSONPreviewVisualize

1{2"success": true,3"message": "Order placed successfully",4"data": {5"orderId": 6,6"userId": 6,7"orderDate": "2026-04-13T11:25:07.1564408Z",8"totalAmount": 160.00,9"items": [10{11"productId": 20,12"productName": "Chocolate Bar",13"quantity": 1,14"price": 120.0015},16{17"productId": 19,18"productName": "Biscuits Pack",19"quantity": 1,20"price": 40.0021}22}23}

Get My Orders

GET: <https://localhost:7148/api/Orders/my-orders>

https://localhost:7148/api/Orders/my-orders

GEThttps://localhost:7148/api/Orders/my-orders

DocsParamsAuthorizationHeaders 7BodyScriptsTestsSettings

Auth TypeBearer TokenToken

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

BodyCookiesHeaders 4Test Results

{ }JSONPreviewVisualize

1{2"success": true,3"message": "Success",4"data": [5{6"orderId": 6,7"userId": 6,8"orderDate": "2026-04-13T11:25:07.1564408Z",9"totalAmount": 160.00,10"items": [11{12"productId": 20,13"productName": "Chocolate Bar",14"quantity": 1,15"price": 120.0016},17{18"productId": 19,19"productName": "Biscuits Pack",20"quantity": 1,21"price": 40.0022}23}24}

Get Order by ID

GET: [https://localhost:7148 /api/Orders/{id}](https://localhost:7148/api/Orders/{id})

HTTP <https://localhost:7148/api/Orders/6> Save

GET <https://localhost:7148/api/Orders/6> Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token: [REDACTED]

Body Cookies Headers 4 Test Results 200 OK • 42 ms • 424 B

```
{
  "success": true,
  "message": "Success",
  "data": {
    "orderId": 6,
    "userId": 6,
    "orderDate": "2026-04-13T11:25:07.1564408",
    "totalAmount": 160.00,
    "items": [
      {
        "productId": 20,
        "productName": "Chocolate Bar",
        "quantity": 1,
        "price": 120.00
      },
      {
        "productId": 19,
        "productName": "Biscuits Pack",
        "quantity": 1,
        "price": 40.00
      }
    ]
  }
}
```

Globals Vault Tools

Get All Orders (Admin)

GET: <https://localhost:7148/api/Orders/all-orders>

HTTP <https://localhost:7148/api/Orders/my-orders> Save Share

GET <https://localhost:7148/api/Orders/my-orders> Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Auth Type: Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token: [REDACTED]

Body Cookies Headers 4 Test Results 200 OK • 92 ms • 426 B

```
{
  "success": true,
  "message": "Success",
  "data": [
    {
      "orderId": 6,
      "userId": 6,
      "orderDate": "2026-04-13T11:25:07.1564408",
      "totalAmount": 160.00,
      "items": [
        {
          "productId": 20,
          "productName": "Chocolate Bar",
          "quantity": 1,
          "price": 120.00
        },
        {
          "productId": 19,
          "productName": "Biscuits Pack",
          "quantity": 1,
          "price": 40.00
        }
      ]
    }
  ]
}
```

Globals Vault Tools

USER APIs (Admin Only)

Get All Users

GET: <https://localhost:7148/api/Users>

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** <https://localhost:7148/api/Users>
- Auth Type:** Bearer Token
- Token:** [Redacted]
- Body:** JSON response

```
2  "success": true,
3  "message": "Users fetched successfully",
4  "data": [
5    {
6      "userId": 1,
7      "userName": "Syam",
8      "emailAddress": "syam@gmail.com",
9      "role": "Customer"
10   },
11   {
12     "userId": 2,
13     "userName": "Saravana",
14     "emailAddress": "saravana@gmail.com",
15     "role": "Admin"
16   },
17   {
18     "userId": 3,
19     "userName": "tharun",
20     "emailAddress": "tharun@gmail.com",
21     "role": "Customer"
22   },
23   {
24     "userId": 4,
```

API RESPONSE FORMAT

Success Response

```
{
  "success": true,
  "message": "Success",
  "data": {}
}
```

Error Response

```
{  
  "success": false,  
  "message": "Error message",  
  "data": null  
}
```

ERROR HANDLING

Handled globally using middleware.

Status Code	Description
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

SECURITY FEATURES

- Password hashing using BCrypt
- JWT-based authentication
- Role-based authorization
- Protected API endpoints

BUSINESS LOGIC

- Validate product existence before ordering
 - Check stock availability
 - Reduce stock after order placement
 - Restrict users to their own orders
 - Allow admin full system access
-

HOW TO RUN THE PROJECT

1. Open Project

- Open in **Visual Studio 2022**

2. Configure Database

Update appsettings.json:

```
"ConnectionStrings": {  
  "DefaultConnection": "Your_SQL_Server_Connection"  
}
```

3. Apply Migrations

Add-Migration InitialCreate

Update-Database

4. Run Application

dotnet run

5. Access Swagger

<https://localhost:<port>/swagger>

6. Test Flow

- 1.Register user
 - 2.Login → Get JWT Token
 - 3.Authorize in Swagger
 - 4.Test APIs
-

REQUEST FLOW

Client → Middleware → Authentication → Authorization → Controller → Service → Repository → Database
→ Response

VALIDATION RULES

- Name: Required, Min 3 characters
- Description: Required, Min 5 characters
- Price: Must be greater than 0
- Quantity: Must be ≥ 1
- Email: Must be valid format

FUTURE ENHANCEMENTS

- Payment Gateway Integration
- Order Status Tracking
- Refresh Token Implementation
- Pagination & Filtering
- Email Notifications

CONCLUSION

ShopEZ provides a robust, scalable, and secure backend architecture for an E-Commerce platform using modern ASP.NET Core practices, ensuring maintainability and real-world applicability.

Developed By:

Name: KORUKONDA SYAM SARAVANA KUMAR

Email: syam261004@gmail.com
